SmartWaiter — AI-Based Smart Waiter System (Project Proposal)

Date: 21/11/2025

Team Name: TimeFlow Labs

Team Members: Tal Lavi, Roei Shalom

1. Problem Identification

The restaurant industry relies heavily on human waitstaff, creating recurring challenges:

- Long waiting times before a waiter arrives at the table.
- Human errors such as misheard orders, forgotten modifications, or missing items.
- Inconsistent service quality across staff members.
- High turnover rates requiring constant recruiting and training.

These issues reduce customer satisfaction, slow down table turnover, and negatively impact business revenue.

Modern AI and voice-based automation can handle these tasks faster, more accurately, and more consistently.

2. Solution Overview

SmartWaiter is an AI-driven ordering system that replaces traditional waiter interaction for routine tasks. Customers scan a table-specific QR code, opening a dedicated mobile interface where they interact with a voice-based assistant.

The system includes:

- Speech-to-Text for capturing orders
- Text-to-Speech for natural responses
- A structured menu database (ingredients, allergens, availability)
- Automatic routing of items: food → kitchen, drinks → bar, requests → runners
- A staff-facing real-time task interface
- A billing interface at the end of the meal
- A feedback engine that learns and improves recommendations over time

SmartWaiter increases accuracy, speed, and efficiency while improving the dining experience.

3. Use Cases

1. Customer Ordering via QR
A customer arrives at the restaurant and sits at a table.
On the table, a unique QR code is displayed.
The customer scans the QR code using their smartphone. The customer is then prompted to wait for a short validation: the shift manager enters the restaurant's security code into the system, verifying that the customer is indeed seated inside the restaurant and linked to the correct table. After validation, the customer is automatically redirected into a dedicated ordering interface specific to that table.
Immediately, a voice-based AI waiter initiates a natural conversation with the customer, greeting them and offering assistance.
The AI waiter can: explain dishes, ask clarifying questions for modifications (e.g "How would you like your steak cooked?") and take the order conversationally
Once the customer finalizes their order, the AI waiter processes it and routes each item automatically - Food items → Kitchen, cocktails and drinks → Bar, service requests → Runners.
During the session, the customer can continue asking questions, order additional items, or request assistance - all through the AI waiter.

2. Runner Task Handling
The customer requests items such as napkins, toothpicks, extra plates, or table clearing.
The AI waiter identifies that the request is relevant to runners and automatically pushes a realtime notification to the runner interface. The runner sees a list of tasks, sorted by urgency and timestamp, each containing table number + short task description. Once the runner completes the task, they mark it as done, and the system updates the customer that the request is being handled or completed.

3. Manager Handling Unavailable Dishes
When the kitchen runs out of a dish, the kitchen alerts the shift manager. The manager updates the system, marking the dish as unavailable. If a customer attempts to order it, the AI waiter checks the menu availability database, informs the customer that the item is no longer available, and intelligently suggests a similar dish (e.g., another appetizer or similar main course).

4. Customer Payment Flow
Upon finishing the meal, the customer asks the AI waiter for the bill. The system displays the full invoice and provides an integrated payment interface. Customers can pay the entire bill or choose partial payment to split costs among diners. After payment is completed, the AI waiter thanks the customer and closes the session.

4.Requirements

Costumer Screen Requirements

- QR onboarding with table specific session
- Manager security code validation
- Voice-based AI conversation engine
- Status List of requests
- Billing interface with full and partial payment options
- Visiting time counter
- Button for human support
- Unified Application Access:
  The system is implemented as a single unified application.
  Upon launch, the user selects the desired role:
  Customer, Staff (Runner / Kitchen / Bar), or Manager.
  Each role provides a dedicated interface and permissions,
  while sharing the same underlying application infrastructure.

Runner/Kitchen/Bar Screen Requirements

- Runner task dashboard with prioritization
- Option for approving and changing status of costumer request

Manager Screen Requirements

- Editing stock availability
- List of table human requests
- Tracking after

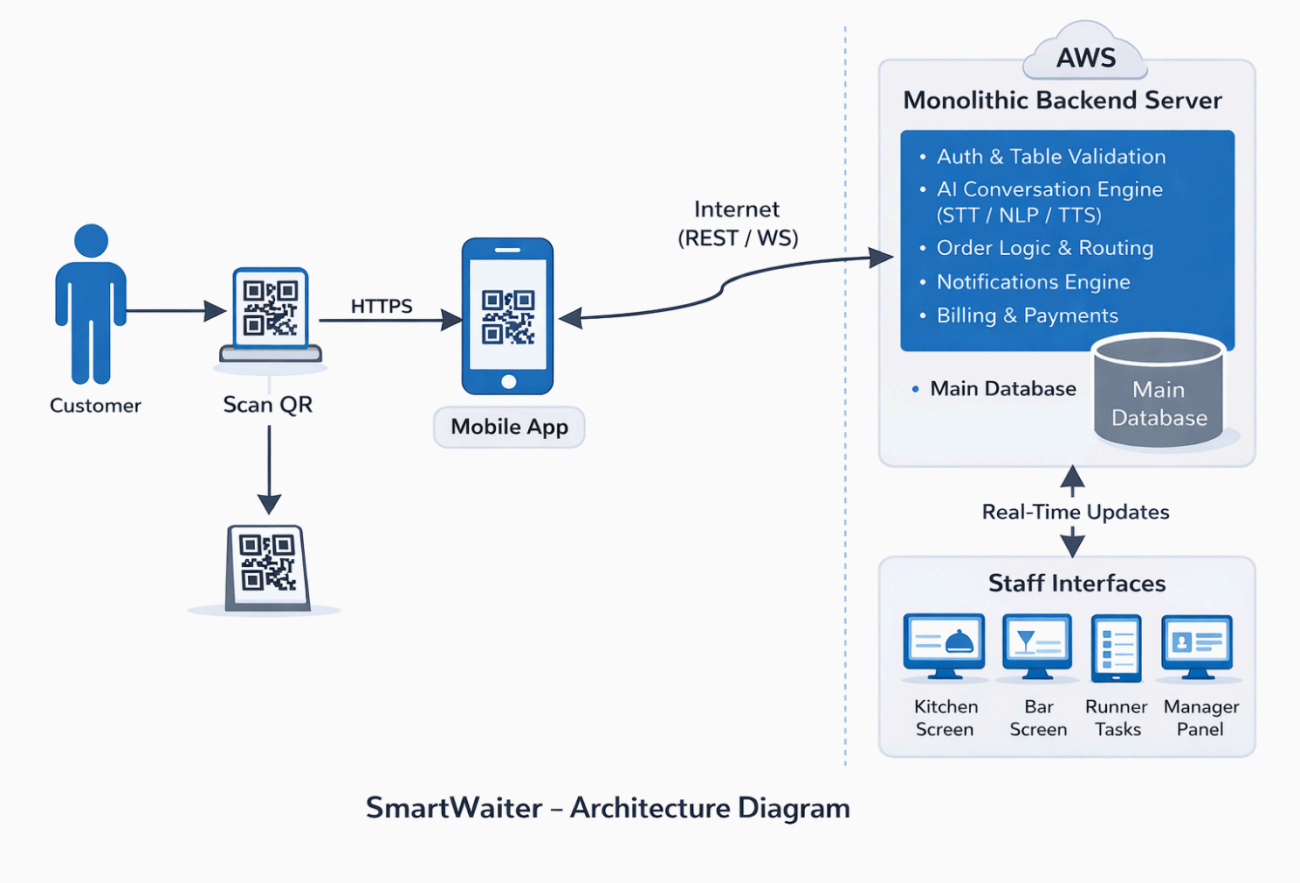
Functional Requirements

- Menu database with ingredients, availability tracking
- Costumer request database for improvement review
- Automatic routing of food orders to kitchen
- Automatic routing of drink orders to bar
- Automatic routing of service tasks to runners
- Real-time notifications between system and staff

Technical Requirements

- Stable mobile web/app interface
- Database for menu, orders, tasks, availability
- Secure communication between client and server

5. System Architecture



SmartWaiter – Architecture Diagram

Modules

The SmartWaiter system is composed of several functional modules, distributed across the frontend application and the backend server.
Each module is responsible for a specific role in the system and is aligned with the system requirements and use cases.

Customer Application Module

- This module represents the customer-facing interface, accessed by scanning a table-specific QR code.

Sub-modules:

- **QR Onboarding & Table Session Module**
  Handles QR code scanning, table identification, and session initialization.
  Includes manager security code validation to ensure the customer is physically located inside the restaurant.

- **Voice Interaction Module**
  Manages voice-based interaction using Speech-to-Text and Text-to-Speech technologies.
  Enables natural conversation, clarification questions, and confirmations during ordering.

- **Menu Exploration & Recommendation Module**
  Retrieves menu data including ingredients, allergens, and availability.
  Suggests alternatives when items are unavailable and provides intelligent recommendations.

- **Order Management Module**
  Collects and structures customer orders, including modifications and special requests.
  Sends finalized orders to the backend for routing.

- **Customer Request Module**
  Handles service-related requests such as napkins, clearing tables, or assistance.
  Displays request status updates in real time.

- **Billing & Payment Module**
  Displays the full invoice, supports partial and split payments, and finalizes the dining session.

Runner Module
This module is designed for restaurant runners responsible for non-food service tasks.

Sub-modules:

- **Task Reception Module**
  Receives service requests routed from the backend in real time.

- **Task Prioritization Dashboard**
  Displays tasks ordered by urgency and timestamp, including table number and request description.

- **Task Status Update Module**
  Allows runners to mark tasks as in progress or completed, triggering real-time updates to customers.

Kitchen Module
This module supports kitchen staff handling food preparation.

Sub-modules:

- **Food Order Queue Module**
  Displays incoming food orders, including dish details and modifications.

- **Order Status Management Module**
  Enables kitchen staff to update preparation status (e.g., received, preparing, ready).

- **Availability Notification Module**
  Notifies the manager when items are out of stock or unavailable.

Bar Module
This module is responsible for handling drink and cocktail orders.

Sub-modules:

- **Drink Order Queue Module**
  Displays drink-specific orders routed from customer sessions.

- **Preparation Status Module**
  Allows bar staff to update drink preparation status.

- **Real-Time Update Module**
  Sends order status updates to the backend for synchronization with customers and runners.

Manager Module
This module provides operational control and monitoring for shift managers.

Sub-modules:

- **Menu Availability Management Module**
  Allows managers to mark items as available or unavailable in real time.

- **Human Assistance Monitoring Module**
  Displays customer requests for human support that require managerial intervention.

- **Operational Tracking Module**
  Tracks active tables, ongoing orders, and system usage metrics.

Backend Core Module
This module serves as the central logic layer of the system.

Sub-modules:

- **Authentication & Role Management Module**
  Manages user roles (Customer, Runner, Kitchen, Bar, Manager) and access permissions.

- **Order Routing Engine**
  Automatically routes food orders to the kitchen, drinks to the bar, and service tasks to runners.

- **Menu & Availability Engine**
  Maintains menu data, ingredient lists, and real-time availability status.

- **Real-Time Communication Module**
  Implements WebSocket-based communication for instant updates between customers and staff.

- **Data Persistence Module**
  Stores orders, requests, sessions, and feedback in the PostgreSQL database.

- **Feedback & Learning Module**
  Collects customer interaction data for future system optimization and recommendation improvement.

6. Technology Stack

Backend - Roei:
Language: typescript
Framework: node.js

Mobile(frontend) - Tal:
Language: typescript
Framework: react native

Database:
Language: SQL
Technology: PostgreSQL (with TimescaleDB extension for time-series data).

Cloud Infrastructure:

The system is deployed on Amazon Web Services (AWS).

AWS is used to host the backend services, database, and real-time communication components, providing scalability, reliability, and secure cloud-based infrastructure.

Backend and frontend communication:
The communication between the frontend and backend is based on a hybrid approach combining REST APIs for client-initiated actions and WebSocket connections for real-time updates. This design enables efficient request handling while ensuring immediate system feedback to customers and staff.

7. API

Customer App API-

Endpoint: GET /api/menu

Description: Retrieves the full structured menu, including dish details, ingredients, and real-time availability.
Request JSON: None (Query parameters can be used for filtering by category, e.g., ?category=food).
Response JSON: menu_items: [{ "id", "name", "description", "price", "category", "is_available", "metadata": {"allergens": [], "ingredients": []} }]

Endpoint: POST /api/orders

Description: Processes a finalized order from the AI interaction and routes items to the kitchen or bar.
Request JSON: table_id, items: [{ "menu_item_id", "quantity", "modifications" }], total_price
Response JSON: order_id, status, estimated_time

Endpoint: POST /api/requests

Description: Handles service-related requests (e.g., napkins, assistance) and routes them to the runner interface.
Request JSON: table_id, request_type (string), priority (optional)
Response JSON: request_id, status ("pending"), timestamp

Staff/Runner/Kitchen API-

Endpoint: GET /api/tasks

Description: Fetches active tasks for staff screens (Kitchen/Bar/Runner) sorted by urgency.
Request JSON: role (enum: "kitchen" | "bar" | "runner")
Response JSON: tasks: [{ "id", "table_id", "description", "status", "created_at", "priority" }]

Endpoint: PATCH /api/tasks/{id}/status

Description: Updates the status of a specific order or service request in real-time.
Request JSON: status (string: "preparing" | "ready" | "completed")
Response JSON: task_id, updated_status, updated_at

Endpoint: PATCH /api/menu/{id}/availability

Description: Allows managers to update the stock status of a dish when it runs out.
Request JSON: is_available (boolean)
Response JSON: menu_item_id, is_available

8. DB Tables

menu_items:

- **Purpose**: The single source of truth for all restaurant offerings, ingredients, and availability.
- **Columns**:
    - id (UUID, PK)
    - name (VARCHAR)
    - description (TEXT)
    - price (DECIMAL)
    - category (VARCHAR): e.g., "food", "drinks".
    - is_available (BOOLEAN).
    - metadata (JSONB): Stores {"ingredients": [], "allergens": []}.

orders:

- **Purpose**: Tracks customer sessions and links multiple items to a specific table.
- **Columns**:
    - id (UUID, PK)
    - table_id (INTEGER)
    - status (VARCHAR): e.g., "active", "paid".
    - created_at (TIMESTAMP)
    - total_price (DECIMAL)

order_items:

- **Purpose**: Stores the specific items and modifications within each order.
- **Columns**:
    - id (UUID, PK)
    - order_id (UUID, FK)
    - menu_item_id (UUID, FK)
    - modifications (TEXT): e.g., "Medium Rare".
    - status (VARCHAR): e.g., "preparing", "ready".

9. Project Timeline and Development Milestones

Phase 1: Infrastructure & Core MVP (January – February 2026)
Goal: Establish the system foundation and achieve a basic "Customer-to-Kitchen" order flow.

- Database & Repository Setup: Initialize the GitHub repository with separate folders for Backend and Mobile. Create the PostgreSQL schema including menu_items and orders tables.
- Core API Development: Implement basic REST endpoints for retrieving the menu and submitting a table-specific order.
- Basic Mobile Interface: Develop the initial React Native screens for QR scanning and menu display.
- Milestone: A customer can scan a code and send a manual order that appears in the database and a basic staff view.

Phase 2: AI Integration & Task Routing (March – April 2026)
Goal: Implement the voice-based interface and automated logic for staff roles.

- AI Conversation Engine: Integrate Speech-to-Text (STT) and Text-to-Speech (TTS) to enable natural ordering dialogue.
- Automated Routing Engine: Develop the backend logic to route food to the kitchen, drinks to the bar, and service requests to runners.
- Staff Dashboards: Build the dedicated interfaces for Runners, Kitchen, and Bar staff to manage their respective queues.

Phase 3: Operations, Billing & Mid-Term Review (May – June 2026)
Goal: Complete the dining cycle and present the system for external evaluation.

- Billing & Payments: Implement the full and partial payment interface.
- Managerial Controls: Develop the "Out of Stock" management module and human assistance monitoring.
- Workshop Presentations: Present the functional prototype during the Workshop Days (May 24 – June 5) for external examiner review.

Phase 4: Optimization & Feedback Loops (July – August 2026)
Goal: Refine the user experience and finalize all technical requirements.

- Learning & Recommendations: Implement the feedback engine that improves dish suggestions over time.
- Real-time Synchronization: Optimize WebSocket connections for instant status updates between all roles.
- Final Locking: Final adjustments to the system before the site lock on August 31, 2026.

Phase 5: Project Exhibition (September 2026)
Goal: Public demonstration of the completed SmartWaiter system.

- Project Exhibition (08.09.2026): Showcasing the final product at the annual Project Fair.

7. Existing Alternatives

Presto Voice (USA):

AI-driven conversational ordering for drive-thru restaurants. Focused on drive-thru automation, not QR-based in-restaurant table service.

Toast:

A leading POS system for restaurants, but lacking voice interaction and conversational AI.

BeeComm (Israel):

Strong operational systems but no customer-facing AI ordering.

Alexa for Hospitality:

Hotel-oriented voice assistant not designed for restaurant order routing or menu intelligence.

SmartWaiter is unique for combining QR onboarding, voice ordering, menu intelligence, routing, upselling AI, and feedback learning into one unified system.

8. User Demographics

Primary Users:

- Restaurant customers (diners) placing voice-based orders and requesting information.

Secondary Users:

- Kitchen staff (food orders)
- Bar staff (drink orders)
- Runners (service tasks)

- Managers (operational insights and optimization)